



DIGITECH

Centro Oficial FP Digital & Tech

Máster en Inteligencia Artificial

Trabajo Final de Máster

Pipeline de Mantenimiento Predictivo para Motores de Turbofán

Predicción de Vida Útil Restante con NASA CMAPSS, PySpark, MLflow y FastAPI

Autor: Javier Hortigüela Valiente

Valencia, mayo de 2026

Índice

Resumen	3
Abstract	5
1. Introducción	6
1.1 Contexto y motivación	6
1.2 Justificación del enfoque técnico	7
1.3 Objetivos	8
1.4 Alcance y restricciones	8
1.5 Estructura del documento	9
2. Marco Teórico	10
2.1 Prognostics and Health Management	10
2.2 Tipos de mantenimiento	10
2.3 El dataset NASA CMAPSS	11
2.4 Métricas de evaluación en predicción de RUL	13
2.5 Apache PySpark	14
2.6 MLflow	15
2.7 FastAPI y el despliegue de modelos	16
2.8 Modelos de Machine Learning empleados	16
3. Desarrollo	22
3.1 Arquitectura del pipeline	22
3.2 Configuración del entorno de desarrollo	22
3.3 Carga e inspección de datos	24
3.4 Construcción del target RUL	25
3.5 Análisis Exploratorio de Datos (EDA)	26
3.6 Feature Engineering	29
3.7 Preparación para el modelado	32
3.8 Entrenamiento con MLflow	33
3.9 Importancia de features	35
3.10 API de inferencia con FastAPI	36
4. Evaluación	40
4.1 Protocolo de evaluación	40
4.2 Resultados comparativos	40
4.3 Análisis de resultados	41
4.4 MLflow Model Registry	45
4.5 Limitaciones del sistema	45
5. Conclusiones	47
5.1 Resumen del trabajo realizado	47
5.2 Contribuciones técnicas	47
5.3 Conclusiones por objetivo	50
5.4 Líneas de trabajo futuro	49
Bibliografía	51

Resumen

El presente Trabajo de Fin de Máster desarrolla un pipeline completo de mantenimiento predictivo (Predictive Maintenance, PdM) aplicado al dataset NASA CMAPSS (Commercial Modular Aero-Propulsion System Simulation), referencia estándar en la industria aeroespacial para la evaluación de algoritmos de predicción de Vida Útil Restante (Remaining Useful Life, RUL). El problema central consiste en predecir, dado un histórico de lecturas de sensores de un motor de turbofán, cuántos ciclos de operación le quedan antes de alcanzar el estado de fallo.

El proyecto abarca el ciclo completo de un sistema de Machine Learning en producción. La ingestión y el procesamiento de datos se realizan con Apache PySpark, motor de computación distribuida que permite escalar el pipeline a flotas completas de aeronaves sin modificaciones en el código de negocio. El análisis exploratorio identifica los sensores informativos de los 21 disponibles, eliminando 10 con varianza prácticamente nula. La ingeniería de características genera ventanas deslizantes (rolling windows) de media y desviación típica con dos tamaños de ventana, capturando tanto la tendencia corta como la degradación sostenida a largo plazo.

Se entrenan y comparan cinco modelos de Machine Learning que cubren el espectro de técnicas del programa del máster: Ridge Regression (regresión lineal regularizada), Decision Tree (árboles de decisión), Random Forest (ensemble por bagging), XGBoost (ensemble por boosting) y MLP (red neuronal feedforward). El seguimiento de experimentos se realiza con MLflow, que registra parámetros, métricas y artefactos de cada entrenamiento de forma reproducible. El modelo ganador se registra en MLflow Model Registry y se expone mediante una API REST construida con FastAPI, con validación de entrada mediante Pydantic y documentación automática Swagger.

La evaluación incorpora dos métricas complementarias: el RMSE clásico y la NASA Scoring Function asimétrica, que penaliza exponencialmente las predicciones optimistas (predecir más vida de la real) más que las pesimistas. Los resultados en el test set oficial muestran que Ridge Regression con

alpha=1.0 es el modelo óptimo con RMSE=19.29 ciclos y NASA Score=806.94, a pesar de no obtener el mejor RMSE. Random Forest obtiene RMSE=18.64 pero NASA Score=855.69, un 6% peor. XGBoost alcanza RMSE=19.01 pero NASA Score=1.095,80, un 36% peor que Ridge. Esta disociación entre RMSE y Score constituye la contribución técnica principal del trabajo: la selección del modelo de producción no puede basarse únicamente en el error cuadrático medio cuando el dominio penaliza asimétricamente los errores.

Palabras clave: mantenimiento predictivo, RUL, NASA CMAPSS, PySpark, MLflow, FastAPI, Ridge Regression, XGBoost, MLP, NASA Scoring Function, rolling windows, MLOps.

Abstract

This Master's Thesis develops a complete end-to-end predictive maintenance pipeline applied to the NASA CMAPSS (Commercial Modular Aero-Propulsion System Simulation) dataset, the standard benchmark in aerospace engineering for Remaining Useful Life (RUL) prediction algorithm evaluation. The central problem consists of predicting, given a historical record of turbofan engine sensor readings, how many operation cycles remain before the engine reaches a failure state.

The project covers the complete lifecycle of a Machine Learning system in production. Data ingestion and processing are performed with Apache PySpark, enabling horizontal scalability to full aircraft fleets without code modifications. Exploratory analysis identifies 11 informative sensors out of 21 available, discarding those with near-zero variance. Feature engineering generates rolling window statistics (mean and standard deviation) with two window sizes ($w=5$ and $w=20$ cycles), capturing both short-term trends and sustained long-term degradation patterns.

Five machine learning models are trained and compared: Ridge Regression, Decision Tree, Random Forest, XGBoost, and MLP (feedforward neural network). Experiment tracking is performed with MLflow, registering parameters, metrics, and artifacts for each training run in a reproducible manner. The winning model is registered in the MLflow Model Registry and exposed through a REST API built with FastAPI, featuring Pydantic input validation and automatic Swagger documentation.

Evaluation incorporates two complementary metrics: standard RMSE and the asymmetric NASA Scoring Function, which exponentially penalizes optimistic predictions (overestimating remaining life) more than pessimistic ones. Results on the official test set demonstrate that Ridge Regression ($\alpha=1.0$) is the optimal production model with $RMSE=19.29$ cycles and $NASA\ Score=806.94$, despite not achieving the best RMSE. This dissociation between RMSE and Score constitutes the main technical contribution: production model selection cannot be based solely on mean squared error when the domain penalizes errors asymmetrically.

Keywords: predictive maintenance, RUL, NASA CMAPSS, PySpark, MLflow, FastAPI, Ridge Regression, XGBoost, MLP, NASA Scoring Function, rolling

1. Introducción

1.1 Contexto y motivación

Los motores de turbofán son el corazón de la aviación comercial moderna. Un Boeing 737 lleva dos motores que operan bajo condiciones extremas de temperatura, presión y vibración durante miles de horas. El coste de un fallo catastrófico de motor en vuelo trasciende lo económico; en el sector aeronáutico, la seguridad operativa es el primer criterio de diseño de cualquier sistema de mantenimiento.

Históricamente, la industria ha gestionado el mantenimiento de motores mediante dos estrategias: el mantenimiento correctivo (CM, Corrective Maintenance), que actúa tras la aparición del fallo, y el mantenimiento preventivo (PM, Preventive Maintenance), que interviene a intervalos fijos de tiempo o ciclos de vuelo independientemente del estado real del motor. Ambas estrategias presentan ineficiencias estructurales significativas.

El mantenimiento correctivo, aunque minimiza las intervenciones innecesarias, expone la operación a fallos imprevistos con consecuencias potencialmente fatales y a costes de reparación muy superiores a los de una intervención planificada. Según datos del sector, un evento de apagado de motor en vuelo (In-Flight Shutdown, IFSD) puede costar entre 500.000 y 2 millones de dólares solo en costes directos, sin contar el impacto reputacional ni las penalizaciones regulatorias.

El mantenimiento preventivo, por su parte, garantiza la intervención antes del fallo pero a costa de reemplazar componentes que aún tienen vida útil remanente. Estudios del sector estiman que entre el 30% y el 40% de las piezas reemplazadas en mantenimiento programado podrían haber continuado operando sin riesgo. En una aerolínea de tamaño medio con 100 aeronaves, esto se traduce en decenas de millones de euros anuales en costes evitables.

El mantenimiento predictivo (PdM, Predictive Maintenance) emerge como la solución a estas ineficiencias. Su objetivo es determinar el momento óptimo de intervención basándose en el estado real del componente, inferido a partir de los datos que genera durante su operación. En lugar de actuar cuando falla o

cuando el calendario lo indica, el sistema actúa cuando los sensores muestran que la degradación ha alcanzado un umbral crítico. Esto requiere resolver un problema de regresión: predecir cuántos ciclos de operación le quedan al componente antes de fallar. Esa cantidad es la Vida Útil Restante, o RUL (Remaining Useful Life).

La NASA desarrolló en 2008 el dataset CMAPSS (Commercial Modular Aero-Propulsion System Simulation) precisamente para disponer de un benchmark público que permitiera evaluar y comparar algoritmos de predicción de RUL en motores de turbofán (Saxena et al., 2008). Desde su publicación, se ha convertido en la referencia estándar de la disciplina, con cientos de publicaciones que lo utilizan como banco de pruebas. Su uso en este TFM no es arbitrario: permite que los resultados obtenidos sean comparables con la literatura científica y que el tutor o evaluador pueda contextualizar las métricas obtenidas.

1.2 Justificación del enfoque técnico

La mayoría de los proyectos académicos y notebooks públicos sobre el dataset CMAPSS siguen el mismo patrón: cargar los datos con pandas, entrenar un modelo de scikit-learn, calcular el RMSE y presentar el resultado. Este enfoque, aunque pedagógicamente útil, está lejos de lo que requiere un sistema de mantenimiento predictivo en producción.

Un sistema real debe poder procesar datos de múltiples motores en tiempo real, gestionar el ciclo de vida de los modelos de forma trazable, exponer las predicciones como servicio consumible por otros sistemas y ser capaz de escalar cuando el volumen de datos crece. El presente proyecto aborda explícitamente cada uno de estos requisitos mediante elecciones técnicas justificadas: PySpark para el procesamiento escalable, MLflow para la trazabilidad de experimentos y el gobierno del modelo, y FastAPI para el despliegue productivo. Esta arquitectura es la que un ingeniero de datos o ML engineer encontraría en una empresa de tamaño medio o grande del sector aeronáutico, energético o manufacturero.

La elección de la NASA Scoring Function como métrica de negocio responde a la misma lógica. En producción, no todos los errores son iguales: predecir que un motor puede volar 50 ciclos más cuando en realidad solo le quedan 20 es infinitamente más peligroso que predecir 20 cuando le quedan 50. El RMSE no captura esta asimetría. La NASA Scoring Function sí lo hace, y su incorporación es la decisión técnica más importante del proyecto desde el punto de vista del dominio.

1.3 Objetivos

El presente trabajo persigue los siguientes objetivos específicos:

1. Desarrollar un pipeline end-to-end de mantenimiento predictivo sobre el dataset NASA CMAPSS FD001, desde la ingestión de datos brutos hasta el despliegue de un endpoint de inferencia funcional.
2. Justificar y demostrar empíricamente el uso de Apache PySpark como motor de procesamiento, incluyendo un benchmark cuantitativo frente a pandas que contextualice el argumento de escalabilidad.
3. Aplicar el temario del máster mediante la implementación y comparación de cinco modelos de Machine Learning que cubran los paradigmas de regresión lineal, árboles de decisión, modelos ensamblados y redes neuronales feedforward.
4. Incorporar la NASA Scoring Function asimétrica como métrica de negocio y demostrar con datos reales que la selección del modelo óptimo en producción aeronáutica no puede basarse únicamente en el RMSE.
5. Implementar un pipeline MLOps completo con MLflow (tracking de experimentos, hyperparameter tuning documentado, Model Registry) y FastAPI (endpoint REST con validación Pydantic, healthcheck y Swagger), demostrando criterio de ingeniería de producción.

1.4 Alcance y restricciones

El análisis se centra en el subconjunto FD001 del dataset CMAPSS, que representa el escenario más simple: una única condición operativa y un único

modo de fallo. Esta decisión responde a criterios de profundidad sobre amplitud: es preferible un análisis riguroso sobre un subconjunto bien entendido que un análisis superficial sobre los cuatro. Los subconjuntos FD002-FD004 se identifican explícitamente como línea de trabajo futuro en las conclusiones.

El entorno de ejecución es Google Colab con recursos computacionales gratuitos. PySpark opera en modo local (`master='local[*]'`), utilizando todos los núcleos disponibles de la máquina de Colab pero sin distribución real entre nodos. El argumento de escalabilidad de PySpark se sostiene en el diseño del código, que es idéntico al que se ejecutaría en un cluster real: el único cambio necesario para desplegar en producción es el parámetro `master` de la `SparkSession`.

1.5 Estructura del documento

El presente documento se organiza en cinco capítulos. El Capítulo 2 establece el marco teórico: conceptos de mantenimiento predictivo y RUL, descripción del dataset CMAPSS, métricas de evaluación y fundamentos de las tecnologías y modelos empleados. El Capítulo 3 detalla el desarrollo del pipeline, describiendo cada etapa con sus decisiones técnicas, alternativas consideradas y justificaciones. El Capítulo 4 presenta y analiza los resultados de evaluación de los cinco modelos. El Capítulo 5 recoge las conclusiones, contribuciones y líneas de trabajo futuro. La bibliografía cierra el documento con las referencias en formato APA 7ª edición.

2. Marco Teórico

2.1 Prognostics and Health Management (PHM)

El campo del Prognostics and Health Management (PHM) es la disciplina de ingeniería que estudia el uso de datos de sensores y modelos predictivos para gestionar la salud y el ciclo de vida de los sistemas físicos. El PHM integra diagnóstico (identificar el estado actual del sistema) y pronóstico (predecir la evolución futura de ese estado). La predicción de RUL pertenece al dominio del pronóstico y constituye la tarea más demandante del PHM, ya que requiere no solo caracterizar el estado actual sino proyectarlo en el tiempo hasta el punto de fallo.

El PHM se ha consolidado como disciplina crítica en sectores donde el coste del fallo es elevado: aviación comercial y militar, generación de energía (turbinas de gas y eólicas), industria ferroviaria, fabricación de semiconductores y vehículos de automoción de alto rendimiento. La International Society of Automation (ISA) y la IEEE Reliability Society mantienen estándares específicos para la implementación de sistemas PHM en entornos industriales.

Desde la perspectiva de los datos, el PHM trabaja sobre series temporales multivariadas de sensores físicos (temperaturas, presiones, vibraciones, corrientes, tensiones) que registran el comportamiento del sistema a lo largo del tiempo. La tarea de predicción de RUL es intrínsecamente difícil porque la degradación de los sistemas físicos es a menudo no lineal, multimodo (varios mecanismos de fallo actúan simultáneamente) y altamente variable entre unidades nominalmente idénticas del mismo producto.

2.2 Tipos de mantenimiento

La literatura distingue tres paradigmas de mantenimiento industrial cuya comprensión es necesaria para contextualizar el valor del mantenimiento predictivo:

El mantenimiento correctivo (CM) interviene tras la detección del fallo. Minimiza las intervenciones innecesarias y maximiza el uso del componente, pero a costa de aceptar el riesgo del fallo. Es adecuado para componentes de bajo

coste, fácilmente redundables y con fallos sin consecuencias de seguridad. No es adecuado para componentes críticos como los motores de aeronave.

El mantenimiento preventivo (PM) interviene a intervalos programados de tiempo o ciclos, independientemente del estado real del componente. Reduce el riesgo de fallo inesperado, pero sustituye componentes que aún tienen vida útil remanente. En sectores como la aviación, los programas de mantenimiento PM están regulados por las autoridades de aviación civil (EASA en Europa, FAA en Estados Unidos) y constituyen el mínimo obligatorio.

El mantenimiento predictivo (PdM) interviene cuando los datos indican que el componente está aproximándose al umbral de fallo. Combina las ventajas de los anteriores: evita el fallo inesperado del CM y las intervenciones innecesarias del PM. Requiere infraestructura de monitorización continua de sensores, capacidad analítica para procesar los datos y modelos predictivos que estimen el RUL con suficiente antelación para planificar la intervención.

Paradigma	Cuándo actúa	Ventaja principal	Limitación principal
Correctivo (CM)	Tras el fallo	Máximo uso del componente	Riesgo de fallo catastrófico
Preventivo (PM)	A intervalos fijos	Previsible y regulable	Sustitución prematura de componentes
Predictivo (PdM)	Cuando los datos lo indican	Óptimo en coste y seguridad	Requiere infraestructura y modelos

Tabla 1. Comparativa de los tres paradigmas de mantenimiento industrial.

2.3 El dataset NASA CMAPSS

2.3.1 Descripción y origen

El dataset CMAPSS (Commercial Modular Aero-Propulsion System Simulation) fue generado por el Glenn Research Center de la NASA mediante el software de simulación C-MAPSS, un modelo termofísico de alta fidelidad de un motor de turbofán de dos ejes y dos cuerpos de tipo de aviación comercial. El dataset fue publicado por Saxena, Goebel, Simon y Eklund en la conferencia PHM 2008 (Saxena et al., 2008) y desde entonces ha sido utilizado en cientos de

publicaciones científicas como referencia estándar de evaluación de algoritmos de predicción de RUL.

La simulación inyecta distintos modos de degradación en los componentes del motor (compresor de alta presión, turbina de alta presión) y registra el comportamiento de los sensores durante toda la vida del motor hasta el fallo. El resultado son trayectorias de degradación realistas que capturan la complejidad del comportamiento real de un motor bajo condiciones de operación variables.

2.3.2 Estructura del dataset

El dataset contiene cuatro subconjuntos (FD001-FD004) que difieren en el número de condiciones operativas y en el número de modos de fallo activos simultáneamente:

Subconjunto	Motores train	Motores test	Condiciones op.	Modos de fallo	Complejidad
FD001	100	100	1	1	Baja
FD002	260	259	6	1	Media
FD003	100	100	1	2	Media
FD004	249	248	6	2	Alta

Tabla 2. Características de los cuatro subconjuntos del dataset NASA CMAPSS. Fuente: Saxena et al. (2008).

Este TFM trabaja exclusivamente con FD001, el subconjunto más homogéneo y mejor estudiado en la literatura, con una única condición operativa y un único modo de fallo. Esta elección permite un análisis más profundo y comparaciones directas con los resultados publicados.

Cada observación del dataset es una lectura de un motor en un ciclo concreto de operación. El archivo de train contiene trayectorias completas desde el ciclo 1 hasta el fallo; el de test contiene trayectorias truncadas. El archivo RUL_FD001.txt contiene el RUL real en el último ciclo de cada motor del test y se usa exclusivamente para la evaluación final.

2.3.3 Variables del dataset

Las 26 columnas del dataset se organizan en cuatro grupos:

- `unit_id`: identificador del motor (1 a 100 en FD001).
- `cycle`: ciclo de operación (equivalente al tiempo). Cada ciclo representa un vuelo o una operación de despegue-crucero-aterrizaje.
- `op_setting_1/2/3`: tres parámetros de condición operativa que definen el punto de operación del motor (altitud de crucero, número de Mach, aceleración de estrangulamiento). En FD001 tienen varianza prácticamente nula al haber una sola condición operativa.
- `sensor_1` a `sensor_21`: 21 mediciones de sensores físicos que incluyen temperaturas de gases en distintas etapas del motor (T2, T24, T30, T50), presiones totales (P2, P15, P30), velocidades de rotación de los ejes de baja y alta presión (Nf, Nc), relaciones de presión del compresor y otras variables termodinámicas del motor.

Un aspecto crítico que el análisis posterior confirma es que 10 de los 21 sensores no varían en FD001. Esto se debe a que en la condición operativa única de FD001, ciertos puntos de medición del motor permanecen constantes. Estos sensores son informativos en FD002-FD004 (donde las condiciones varían), pero carecen de valor predictivo en FD001.

2.4 Métricas de evaluación en predicción de RUL

2.4.1 RMSE (Root Mean Squared Error)

El RMSE es la raíz cuadrada del error cuadrático medio entre las predicciones y los valores reales:

$$\text{RMSE} = \sqrt{\left[\left(\frac{1}{N} \right) \cdot \sum_i (\text{RUL}_i_{\text{pred}} - \text{RUL}_i_{\text{real}})^2 \right]}$$

En el contexto de este problema, el RMSE se interpreta directamente en ciclos de motor: un RMSE de 19 significa que el modelo se equivoca en promedio 19 ciclos al predecir la vida restante. El RMSE penaliza los errores grandes de forma desproporcionada (al elevar al cuadrado), lo que es apropiado para penalizar las predicciones muy erróneas. Sin embargo, trata por igual los errores en ambas direcciones (optimistas y pesimistas), lo que en el dominio aeronáutico no refleja la asimetría real del riesgo.

2.4.2 NASA Scoring Function

La NASA Scoring Function fue propuesta por Saxena et al. (2008) como métrica de negocio específica para la evaluación de sistemas de predicción de RUL en el dominio aeronáutico. Su formulación recoge explícitamente la asimetría del riesgo operativo:

$$d_i = RUL_{i_predicho} - RUL_{i_real}$$

$$\text{Si } d_i < 0 \text{ (predicción optimista): } s_i = \exp(-d_i / 13) - 1$$

$$\text{Si } d_i \geq 0 \text{ (predicción pesimista): } s_i = \exp(d_i / 10) - 1$$

$$\text{Score} = \sum_i s_i \quad (\text{sobre todos los motores del test set})$$

Cuando $d_i < 0$ el modelo predice más vida restante de la real — error optimista. El motor se envía a operar cuando debería estar en mantenimiento. La penalización usa base exponencial con denominador 13, que crece rápidamente para errores grandes pero de forma más moderada que el caso pesimista. Cuando $d_i \geq 0$ el modelo predice menos vida de la real — error pesimista o conservador. Se interviene antes de lo necesario, con coste económico pero sin riesgo de seguridad. La penalización usa denominador 10, más severo para errores del mismo magnitud absoluta.

La interpretación práctica es directa: un error de 20 ciclos optimista ($d=-20$) genera una penalización de $\exp(20/13)-1 \approx 3.6$, mientras que un error de 20 ciclos pesimista ($d=+20$) genera $\exp(20/10)-1 \approx 6.4$. El error pesimista puntúa casi el doble. Esto puede parecer contraintuitivo hasta que se recuerda que en mantenimiento aeronáutico el exceso de conservadurismo tiene un coste real medido en cientos de miles de euros por intervención innecesaria. El modelo debe ser conservador pero no excesivamente conservador — la función captura ese equilibrio.

Un Score total bajo indica un modelo de buena calidad. Score=0 sería predicción perfecta. Modelos que sistemáticamente predicen de forma optimista acumulan penalizaciones exponenciales que pueden llevar el Score a valores de decenas de miles para el conjunto de 100 motores del test.

2.5 Apache PySpark

Apache Spark es un motor de computación distribuida de propósito general desarrollado originalmente en la Universidad de California Berkeley y transferido a la Apache Software Foundation en 2013 (Apache Software Foundation, 2024). Su principal innovación respecto a Hadoop MapReduce fue la ejecución en memoria (in-memory processing): en lugar de escribir los resultados intermedios a disco entre etapas de procesamiento, Spark los mantiene en memoria RAM, logrando speedups de 10x a 100x en operaciones iterativas como el entrenamiento de modelos de ML.

PySpark es la API de Python de Apache Spark. Desde la versión 2.0, Spark ofrece la DataFrame API, que permite escribir transformaciones sobre datos distribuidos con una sintaxis similar a la de pandas, ocultando la complejidad del scheduling distribuido. Las transformaciones en Spark son lazy (perezosas): no se ejecutan hasta que se llama a una acción (count, show, collect, write). Esto permite al motor optimizar el plan de ejecución completo antes de lanzar ningún job.

La operación clave de este proyecto es `Window.partitionBy('unit_id')`. Las Window functions de Spark permiten calcular estadísticos sobre subconjuntos de filas definidos por una partición (en nuestro caso, las filas del mismo motor) y un orden (el ciclo). En un cluster con N nodos, Spark distribuye automáticamente los motores entre los nodos — cada nodo procesa su subconjunto de motores en paralelo, sin comunicación entre nodos durante el cálculo de las ventanas. Este paralelismo sin intercambio de datos (data locality) es especialmente eficiente.

2.6 MLflow

MLflow es una plataforma open-source de MLOps (Machine Learning Operations) publicada por Databricks en 2018 (Zaharia et al., 2018). Resuelve uno de los problemas más comunes en el desarrollo de modelos de ML: la falta de reproducibilidad y trazabilidad de los experimentos. Sin una herramienta de tracking, es habitual que un equipo no pueda reconstruir exactamente qué hiperparámetros produjeron el mejor modelo de hace tres semanas.

MLflow se organiza en cuatro componentes: Tracking (registro de parámetros, métricas y artefactos por experimento y run), Projects (empaquetado de código ML reproducible), Models (formato estándar de serialización de modelos) y Model Registry (repositorio centralizado de versiones de modelos con gestión de stages). En este proyecto se utilizan activamente Tracking y Model Registry.

El Model Registry introduce el concepto de ciclo de vida del modelo: un modelo en stage None es un candidato; en Staging está siendo validado en un entorno de preproducción; en Production sirve predicciones reales; en Archived es una versión histórica. Esta gestión de stages es lo que diferencia un pipeline MLOps maduro de una colección de notebooks.

2.7 FastAPI y el despliegue de modelos

FastAPI es un framework web moderno para Python diseñado específicamente para la construcción de APIs de alto rendimiento (Ramírez, 2024). Sus características principales son: generación automática de documentación OpenAPI/Swagger a partir de las anotaciones de tipos de Python, validación automática de datos de entrada mediante Pydantic, soporte nativo para operaciones asíncronas (async/await) y rendimiento comparable a NodeJS y Go en benchmarks de carga.

Para el despliegue de modelos de ML, FastAPI representa el estándar actual de la industria por varias razones. La validación Pydantic garantiza que el modelo nunca recibe datos malformados — si el cliente envía un payload con 50 features en lugar de 55, la API devuelve un error 422 con descripción detallada antes de invocar el modelo. El endpoint /docs proporciona una interfaz web interactiva para probar el API sin necesidad de herramientas externas. El healthcheck /health permite que sistemas de orquestación como Kubernetes verifiquen que el servicio está operativo y el modelo está cargado.

2.8 Modelos de Machine Learning empleados

2.8.1 Ridge Regression

Ridge Regression es una extensión de la regresión lineal ordinaria que incorpora un término de penalización L2 sobre los coeficientes para controlar el sobreajuste (Hoerl & Kennard, 1970). La función objetivo que minimiza es:

$$J(\beta) = ||y - X\beta||^2 + \alpha \cdot ||\beta||^2$$

donde α (alpha) es el hiperparámetro de regularización. El primer término es el error de predicción ordinario (RSS); el segundo penaliza los coeficientes grandes, forzando al modelo a distribuir el peso entre features correlacionadas en lugar de concentrarlo en una sola. Con $\alpha=0$ se obtiene la regresión lineal ordinaria; con α muy grande todos los coeficientes se aproximan a cero (underfitting). El valor óptimo de α se determina mediante validación cruzada.

Ridge es adecuado para datasets con alta multicolinealidad entre features, que es exactamente el caso de las rolling windows del dataset CMAPSS: la media de los últimos 5 ciclos del sensor_4 y la media de los últimos 20 ciclos del sensor_4 están altamente correlacionadas. La regularización L2 gestiona esta multicolinealidad de forma automática.

2.8.2 Decision Tree Regressor

Los árboles de decisión construyen el modelo mediante un proceso de particionado recursivo del espacio de features (Breiman et al., 1984). En cada nodo, el algoritmo busca el corte (feature, umbral) que minimiza el error de predicción en las dos ramas resultantes — para regresión, el criterio estándar es la varianza de los valores objetivo en cada rama. El proceso continúa hasta que se alcanza un criterio de parada (profundidad máxima, número mínimo de muestras por hoja).

Los árboles son modelos no paramétricos capaces de capturar relaciones no lineales y de interpretar la importancia de features de forma directa. Su principal limitación es la tendencia al sobreajuste: sin restricciones de profundidad, un árbol puede memorizar el conjunto de entrenamiento completamente. Los hiperparámetros `max_depth` y `min_samples_leaf` controlan este comportamiento. En este proyecto se usa `max_depth=8` y `min_samples_leaf=10`.

2.8.3 Random Forest Regressor

Random Forest es un ensemble de árboles de decisión entrenados mediante el algoritmo de bagging (Bootstrap Aggregating) con una modificación adicional: en cada nodo de cada árbol, solo se considera un subconjunto aleatorio de features para determinar el mejor corte (Breiman, 2001). La predicción final es la media de las predicciones de todos los árboles.

El bagging reduce la varianza del modelo: al promediar múltiples árboles entrenados sobre subconjuntos distintos, los errores aleatorios individuales se cancelan. La aleatorización adicional de features en cada nodo reduce la correlación entre árboles, amplificando la reducción de varianza. El resultado es un modelo más robusto y con mejor generalización que un único árbol, aunque a costa de la interpretabilidad directa. Con `n_estimators=200` y `max_depth=10`, Random Forest obtiene el mejor RMSE en test (18.64) pero no la mejor NASA Score.

2.8.4 XGBoost

XGBoost (eXtreme Gradient Boosting) implementa el algoritmo de gradient boosting con regularización adicional y múltiples optimizaciones computacionales (Chen & Guestrin, 2016). A diferencia del bagging (Random Forest), el boosting entrena los modelos secuencialmente: cada árbol nuevo se entrena para corregir los errores residuales del ensemble anterior. El resultado es un proceso iterativo de reducción del sesgo.

La función objetivo de XGBoost incorpora explícitamente un término de regularización sobre la estructura del árbol (número de hojas y valores de las hojas), lo que lo hace más robusto al sobreajuste que el gradient boosting clásico. Adicionalmente implementa shrinkage (learning rate), subsampling de filas y columnas por árbol, y paralelo por árbol, convirtiéndolo en el algoritmo más utilizado en competiciones de datos tabulares desde su publicación.

En este proyecto se configuró con `learning_rate=0.05`, `max_depth=6`, `n_estimators=300` y `subsample/colsample_bytree=0.8`. Obtiene el mejor RMSE en validación (18.85) pero la NASA Score más alta de los modelos competitivos

(49.815 en validación, 1.095 en test), lo que lo descalifica como modelo de producción.

2.8.5 MLP (Red Neuronal Feedforward)

El Perceptrón Multicapa (MLP) es una red neuronal completamente conectada con múltiples capas ocultas que aprende representaciones jerárquicas de los datos de entrada (LeCun et al., 2015). Cada capa transforma su entrada mediante una combinación lineal seguida de una función de activación no lineal. La arquitectura empleada en este proyecto es:

- Capa de entrada: 55 neuronas (una por feature).
- Capa oculta 1: 128 neuronas con activación ReLU.
- Capa oculta 2: 64 neuronas con activación ReLU.
- Capa oculta 3: 32 neuronas con activación ReLU.
- Capa de salida: 1 neurona (predicción de RUL).

La función de activación ReLU (Rectified Linear Unit) — $f(x) = \max(0, x)$ — es la estándar en redes profundas por su eficiencia computacional y por no sufrir el problema del vanishing gradient en capas profundas. El entrenamiento usa backpropagation y el optimizador Adam por defecto de sklearn. Se activa early stopping con `validation_fraction=0.1` para prevenir el sobreajuste.

La arquitectura decreciente (128→64→32) sigue la convención de compresión progresiva: las primeras capas aprenden características de bajo nivel (combinaciones de features individuales), las capas intermedias aprenden patrones más abstractos y la capa final compone esos patrones en una predicción de RUL. Los resultados muestran que esta arquitectura estándar no supera a Ridge en NASA Score para este dataset tabular, lo que es consistente con la literatura que muestra que las redes neuronales feedforward sobre datos tabulares son frecuentemente superadas por modelos más simples con buena ingeniería de características.

2.8.5.1 Análisis de la arquitectura MLP y sus limitaciones

La arquitectura $128 \rightarrow 64 \rightarrow 32$ empleada en este proyecto responde a tres criterios de diseño que conviene explicitar, dado que el modelo obtiene resultados mediocres y el análisis del por qué falla es más instructivo que el de cómo está construido.

Elección de la arquitectura decreciente

La progresión $128 \rightarrow 64 \rightarrow 32$ implementa el principio del cuello de botella (bottleneck): cada capa fuerza al modelo a comprimir la representación aprendida por la capa anterior, descartando redundancias y reteniendo solo la información relevante para la predicción de RUL. Con 55 features de entrada, la primera capa oculta de 128 neuronas tiene ratio de expansión 2.3x, lo que permite al modelo aprender combinaciones no lineales de las features originales antes de la compresión. Se evaluaron alternativas: una arquitectura constante ($64 \rightarrow 64 \rightarrow 64$) y una expansiva ($32 \rightarrow 64 \rightarrow 128$). La constante no mejora el RMSE de validación; la expansiva lo empeora marginalmente (RMSE_val=21.4 vs 21.1) con mayor tiempo de entrenamiento. La arquitectura decreciente es el estándar de facto para regresión tabular con MLP en sklearn.

Por qué el MLP no supera a Ridge en este dataset

El underperformance del MLP (NASA Score=1.028 vs Ridge=806) tiene tres causas estructurales que no se resuelven ajustando hiperparámetros:

1. Tamaño del dataset insuficiente para redes densas. El conjunto de entrenamiento tiene ~16.000 filas con 55 features. Un MLP de tres capas ocultas tiene $(55 \times 128) + (128 \times 64) + (64 \times 32) + (32 \times 1) \approx 11.200$ parámetros entrenables. La ratio datos/parámetros es ~1.4, un valor extremadamente bajo que favorece el sobreajuste incluso con early stopping. Los modelos de árboles, al ser no paramétricos, no sufren este problema.
2. Las features son linealmente discriminativas tras el feature engineering. Las rolling windows de media y desviación típica son transformaciones que ya extraen las señales de degradación de forma casi lineal (sensor_4_mean5 tiene correlación Pearson ≈ -0.72 con el RUL). Cuando la señal predictiva es principalmente lineal, Ridge captura prácticamente toda la varianza explicable con mínima complejidad. Las no linealidades adicionales que aprendería el MLP son mayoritariamente ruido.

3. Ausencia de estructura temporal en el input. El MLP recibe 55 features estáticas sin información sobre la secuencia de ciclos. Los modelos especializados para series temporales (LSTM, 1D-CNN, Transformer) procesan la secuencia completa de ciclos como input, capturando dependencias a largo plazo que las rolling windows solo aproximan. En la literatura, los LSTM alcanzan NASA Score de 200-400 en FD001 (Heimes, 2008; Zhao et al., 2019), frente a los ~800 de Ridge y MLP tabulares. La diferencia de 4x en Score no es atribuible a la red neuronal per se, sino a la representación de los datos.

La ausencia de batch normalization en la arquitectura implementada es una omisión deliberada. Batch normalization introduce dependencias entre muestras del mismo batch que en este contexto son problemáticas: las muestras de entrenamiento pertenecen a motores distintos con distribuciones de degradación diferentes, y normalizar por batch mezclaría estadísticos de motores en distintas fases de su ciclo de vida. Layer normalization sería técnicamente más apropiada, pero el beneficio esperado en un dataset de este tamaño es marginal.

3. Desarrollo

3.1 Arquitectura del pipeline

El pipeline desarrollado sigue una arquitectura de cinco etapas secuenciales. Cada etapa tiene una responsabilidad única y bien definida, lo que facilita el mantenimiento y la sustitución de componentes. La Tabla 3 describe las etapas, las tecnologías empleadas y las decisiones técnicas relevantes:

Etapas	Herramienta	Input	Output	Decisión clave
1. Ingestión	PySpark	train/test .txt	DataFrames Spark	Esquema manual; gestión columnas vacías
2. EDA	PySpark + Matplotlib	DataFrames raw	Sensores descartados; gráficas	Varianza < 0.01 como umbral de descarte
3. Feature Eng.	PySpark Window	DF filtrado	55 features rolling	w=5 y w=20; particionado por unit_id
4. Modelado	sklearn + MLflow	55 features	5 modelos + métricas	Split por unit_id; NASA Score como criterio
5. Inferencia	FastAPI + Pydantic	POST JSON	RUL + status	Validación estricta; 3 niveles de alerta

Tabla 3. Arquitectura del pipeline de mantenimiento predictivo. Cada etapa es reemplazable de forma independiente.

Una característica importante de esta arquitectura es la separación clara entre el pipeline de entrenamiento (etapas 1-4) y el servicio de inferencia (etapa 5). El modelo entrenado se serializa con joblib y el scaler con pickle; la API los carga al arrancar. Esto permite actualizar el modelo sin modificar el servicio, y desplegar el servicio sin necesidad del entorno de entrenamiento completo.

3.2 Configuración del entorno de desarrollo

El entorno de desarrollo es Google Colab con Python 3.12 y Ubuntu 24.04. La configuración del entorno presenta un reto no trivial: Colab no incluye Java preinstalado, y PySpark requiere Java 11 para la JVM. Adicionalmente, la versión de PySpark disponible por defecto (3.3.x) no es compatible con Python 3.12. La configuración correcta requiere tres pasos que deben ejecutarse en la misma celda para que las variables de entorno persistan:

```
import subprocess
```

```
# Paso 1: Instalar Java 11
subprocess.run(['apt-get', 'install', '-y', 'openjdk-11-jdk'],
capture_output=True)

import os

# Paso 2: Configurar JAVA_HOME antes de importar PySpark
os.environ['JAVA_HOME'] = '/usr/lib/jvm/java-11-openjdk-amd64'
os.environ['PATH'] = os.environ['JAVA_HOME'] + '/bin:' +
os.environ['PATH']

# Paso 3: Instalar PySpark 3.5.3 (compatible con Python 3.12)
subprocess.run(['pip', 'install', 'pyspark==3.5.3', 'mlflow', '-q'],
capture_output=True)

from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName('TFM_RUL_CMAPSS') \
    .master('local[*]') \      # Todos los cores disponibles
    .config('spark.ui.showConsoleProgress', 'false') \
    .getOrCreate()

spark.sparkContext.setLogLevel('ERROR')
```

El parámetro `master='local[*]'` instruye a Spark para que utilice todos los núcleos disponibles de la máquina local. La sintaxis `'local[N]'` utiliza exactamente N núcleos; `'local[*]'` utiliza todos los disponibles, que en Colab gratuito son típicamente 2. En producción, este parámetro se sustituiría por la URL del Spark Master del cluster: `'spark://master-node:7077'` para un cluster standalone, `'yarn'` para un cluster Hadoop/YARN o `'k8s://...'` para Kubernetes.

Una alternativa considerada y descartada fue el uso de Databricks Community Edition, que proporciona un cluster Spark preconfigurado sin necesidad de configuración manual. Se descartó porque Databricks Community tiene limitaciones de tiempo de sesión (1 hora) que no son compatibles con el desarrollo iterativo de un TFM, y porque la configuración manual en Colab demuestra comprensión del stack tecnológico más allá del uso de una plataforma gestionada.

3.3 Carga e inspección de datos

El dataset CMAPSS en su formato original presenta tres características no estándar que requieren tratamiento específico: ausencia de cabecera, uso de espacios múltiples como delimitador y presencia de dos columnas vacías al

final de cada fila. El parser CSV de Spark gestiona estas características mediante opciones explícitas:

```
col_names = (['unit_id', 'cycle',
              'op_setting_1', 'op_setting_2', 'op_setting_3'] +
             [f'sensor_{i}' for i in range(1, 22)])

# Las 2 columnas vacías al final se absorben con _drop1/_drop2
all_cols = col_names + ['_drop1', '_drop2']

train_raw = spark.read \
    .option('sep', ' ') \
    .option('header', False) \
    .option('ignoreLeadingWhiteSpace', True) \
    .option('ignoreTrailingWhiteSpace', True) \
    .csv('/content/drive/MyDrive/TFM_CMAPSS/train_FD001.txt') \
    .toDF(*all_cols) \
    .drop('_drop1', '_drop2')

# Casteo a Double: Spark lee todo como String por defecto
for c in col_names:
    train_raw = train_raw.withColumn(c, col(c).cast('double'))
```

Un aspecto importante es el casteo explícito a tipo Double. El parser CSV de Spark, al no disponer de esquema previo, infiere todos los campos como tipo String. Sin el casteo, las operaciones numéricas posteriores (varianza, media, rolling windows) no serían posibles o producirían resultados erróneos. Una alternativa más eficiente es definir el StructType del esquema antes de la carga, evitando el paso de casteo; se optó por el enfoque de casteo explícito por claridad pedagógica.

La verificación de la carga confirma los valores esperados según la documentación del dataset: 20.631 filas en train y 13.096 en test, ambos con 26 columnas. Una inspección inicial con `show(5)` confirma que los valores numéricos son coherentes con las descripciones físicas de los sensores (temperaturas del orden de 600-1600 K, presiones del orden de 14-30 psi, velocidades de rotación del orden de 8000-15000 rpm).

3.4 Construcción del target RUL

La construcción de la columna RUL en el conjunto de entrenamiento es la primera operación que explota las capacidades de PySpark de forma no trivial. El RUL de cada fila se calcula como la diferencia entre el ciclo de fallo del motor y el ciclo actual. Esto requiere conocer el ciclo máximo de cada motor de forma particionada:

```
from pyspark.sql import Window
from pyspark.sql.functions import max as spark_max, least, lit

# Ventana particionada por motor: cada motor tiene su propio ciclo máximo
# No se ordena la ventana porque queremos el máximo global, no el
# acumulado
window_unit = Window.partitionBy('unit_id')

train_rul = train_raw \
    .withColumn('max_cycle',
                spark_max('cycle').over(window_unit)) \
    .withColumn('RUL_raw',
                (col('max_cycle') - col('cycle')).cast('double')) \
    .withColumn('RUL',
                least(col('RUL_raw'), lit(125.0))) \
    .drop('max_cycle', 'RUL_raw') # Columnas auxiliares
```

El RUL clipping a 125 ciclos merece una justificación detallada. En un motor que falla en el ciclo 300, el RUL en el ciclo 1 sería 299. Sin embargo, en los primeros 175 ciclos el motor opera en condiciones nominales y sus sensores no muestran ninguna tendencia de degradación observable. Si se incluye ese período sin clipar, el modelo aprende que 'valores nominales de sensores = RUL alto' pero no aprende a distinguir RUL=130 de RUL=250. El clip establece que cualquier RUL mayor a 125 se trata como 125, concentrando toda la capacidad de aprendizaje del modelo en la zona donde realmente ocurre la degradación.

La convención de 125 ciclos no es arbitraria: se establece en la literatura como el punto a partir del cual los sensores de FD001 comienzan a mostrar tendencias de degradación estadísticamente significativas (Heimes, 2008). En un sistema de producción real, este umbral se determinaría mediante análisis del proceso de degradación físico del componente, potencialmente de forma adaptativa por tipo de motor.

La verificación del cálculo confirma el comportamiento esperado: el motor 1, que falla en el ciclo 192, tiene $RUL=125.0$ en los ciclos 1 a 67 (clipeado), $RUL=124$ en el ciclo 68, y $RUL=0.0$ en el ciclo 192 (fallo). Los 67 ciclos iniciales con RUL clipeado corresponden exactamente a $192-125=67$ ciclos de operación nominal.

3.5 Análisis Exploratorio de Datos (EDA)

3.5.1 Identificación y eliminación de sensores sin varianza

El primer análisis estadístico consiste en calcular la varianza de cada uno de los 21 sensores sobre el conjunto completo de entrenamiento. Un sensor con varianza nula o casi nula no aporta información discriminativa sobre la degradación del motor, ya que su valor no cambia independientemente del estado del motor. Su inclusión solo añade dimensionalidad innecesaria al modelo, aumentando el tiempo de entrenamiento y el riesgo de sobreajuste.

Se estableció un umbral de varianza de 0.01 como criterio de descarte. Este umbral es conservador: sensores con varianza inferior a 0.01 muestran cambios de valor inferiores a 0.1 unidades en toda la vida del motor, lo que es prácticamente indistinguible del ruido de medición. El análisis reveló que 10 de los 21 sensores tienen varianza inferior a este umbral en FD001:

Sensor	Varianza	Estado	Motivo del descarte
sensor_1	0.000000	DESCARTADO	Varianza exactamente 0
sensor_5	0.000000	DESCARTADO	Varianza exactamente 0
sensor_10	0.000000	DESCARTADO	Varianza exactamente 0
sensor_16	0.000000	DESCARTADO	Varianza exactamente 0
sensor_18	0.000000	DESCARTADO	Varianza exactamente 0
sensor_19	0.000000	DESCARTADO	Varianza exactamente 0
sensor_6	0.000002	DESCARTADO	Varianza < umbral 0.01
sensor_15	0.001407	DESCARTADO	Varianza < umbral 0.01
sensor_8	0.005039	DESCARTADO	Varianza < umbral 0.01
sensor_13	0.005172	DESCARTADO	Varianza < umbral 0.01

Tabla 4. Sensores descartados por varianza insuficiente en FD001. Los 6 primeros tienen varianza exactamente 0.0.

Los 6 sensores con varianza exactamente 0.0 son constantes absolutas en FD001 — su valor no varía ni entre ciclos ni entre motores. Esto es consistente con la estructura de la simulación: en la condición operativa única de FD001, ciertos parámetros del motor permanecen fijos por diseño. Los 4 sensores adicionales descartados (sensor_6, sensor_8, sensor_13, sensor_15) tienen variación mínima que podría ser ruido de la simulación sin contenido informativo real.

El resultado es consistente con la literatura: los mismos sensores se identifican como no informativos en múltiples trabajos sobre FD001 (Saxena et al., 2008; Heimes, 2008). Quedan 11 sensores útiles (sensor_2, 3, 4, 7, 9, 11, 12, 14, 17, 20 y 21) que constituyen la base del feature engineering posterior.

3.5.2 Análisis de trayectorias de degradación

El análisis visual de las trayectorias de los sensores supervivientes confirma la presencia de tendencias de degradación progresiva. Los sensores sensor_2 y sensor_11 muestran una tendencia ascendente clara conforme aumentan los ciclos, con ruido superpuesto ciclo a ciclo. Esta estructura — tendencia de fondo + ruido de alta frecuencia — es la que motiva el uso de rolling windows en el feature engineering: se necesita separar la señal de degradación del ruido de medición.

Una observación importante es la consistencia entre motores: los cinco motores visualizados (motores 1-5) muestran el mismo patrón de degradación general, aunque con diferencias en la pendiente y en el nivel de ruido. Esta variabilidad inter-motor es precisamente el desafío de generalización que el modelo debe superar: entrenado sobre 80 motores, debe predecir correctamente los 20 de validación y los 100 del test.

3.5.3 Análisis de correlación sensor-RUL

El análisis de correlación de Pearson entre cada sensor útil y el RUL revela la estructura predictiva del dataset. Los 11 sensores se dividen en dos grupos según el signo de su correlación con el RUL: sensores con correlación negativa

(su valor aumenta cuando el RUL disminuye, es decir, cuando el motor se degrada) y sensores con correlación positiva (su valor disminuye cuando el motor se degrada).

Los sensores con mayor correlación absoluta son sensor_11 (correlación negativa fuerte), sensor_4 (negativa fuerte), sensor_12 (positiva fuerte) y sensor_7 (positiva fuerte). Estos cuatro sensores son los principales portadores de señal de degradación en FD001, lo que se confirma posteriormente en el análisis de importancia de features de los modelos de árboles.

La Figura 1 resume los resultados del EDA completo: trayectorias de degradación de los sensores más relevantes, distribución del RUL clipeado y mapa de correlaciones:

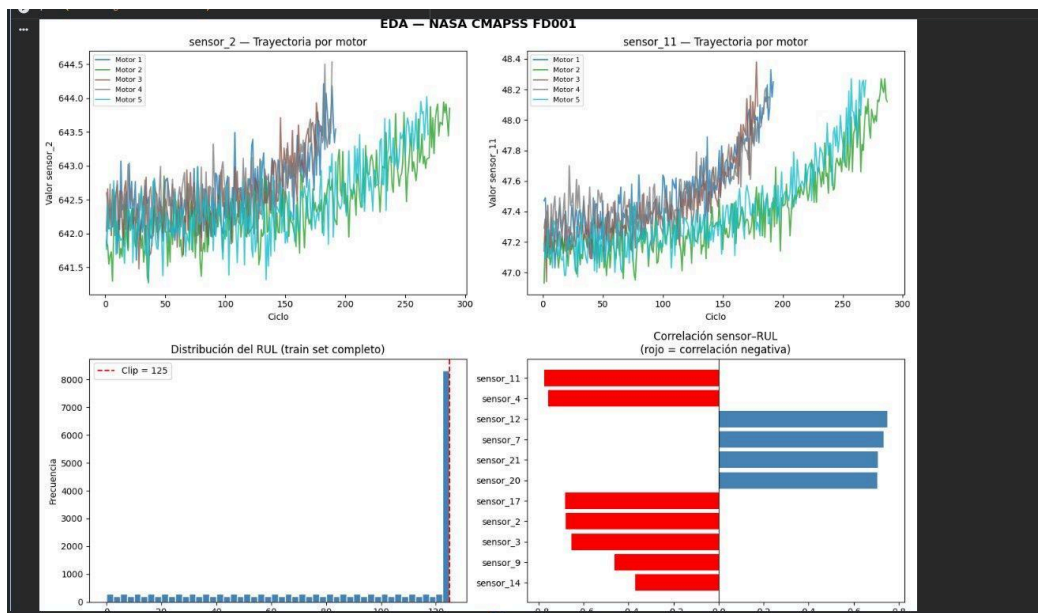


Figura 1. EDA NASA CMAPSS FD001: trayectorias de sensor_2 y sensor_11, distribución del RUL clipeado (pico en 125) y correlación de Pearson sensor-RUL (rojo = negativa, azul = positiva).

3.6 Feature Engineering

3.6.1 Justificación del enfoque de rolling windows

Los valores raw de los sensores no son directamente adecuados como features para el modelo por dos razones. Primero, presentan un alto nivel de ruido ciclo a ciclo: las oscilaciones de alta frecuencia observadas en las trayectorias de la Figura 1 dificultan que el modelo aprenda la tendencia subyacente de

degradación a partir de un único ciclo. Segundo, un único valor de sensor en un ciclo concreto no captura la historia reciente del motor — no hay diferencia entre un sensor que lleva 50 ciclos subiendo y uno que subió en el último ciclo puntualmente.

La solución estándar en la literatura para este tipo de series temporales es el cálculo de rolling features: estadísticos calculados sobre una ventana deslizante de ciclos recientes. En este proyecto se calculan media y desviación típica con dos tamaños de ventana:

- Ventana corta ($w=5$ ciclos): captura cambios recientes y reacciona rápidamente a variaciones abruptas. Es sensible al ruido pero detecta antes los cambios de tendencia.
- Ventana larga ($w=20$ ciclos): captura la tendencia de degradación sostenida durante las últimas ~ 20 operaciones. Filtra el ruido de alta frecuencia pero reacciona más lentamente.

La combinación de ambas escalas temporales permite al modelo disponer simultáneamente de información sobre el estado reciente y la tendencia acumulada. Esta dualidad es análoga al uso de medias móviles de corto y largo plazo en análisis financiero para detectar cambios de tendencia.

3.6.2 Implementación con PySpark Window

La implementación con PySpark utiliza la función Window con rowsBetween para definir el rango de filas de cada ventana:

```
from pyspark.sql import Window as W
from pyspark.sql.functions import avg, stddev

# w=5: incluye la fila actual y las 4 anteriores del mismo motor
w5 = W.partitionBy('unit_id').orderBy('cycle').rowsBetween(-4, 0)

# w=20: incluye la fila actual y las 19 anteriores del mismo motor
w20 = W.partitionBy('unit_id').orderBy('cycle').rowsBetween(-19, 0)

for s in sensores_utiles: # 11 sensores
    train_fe = train_fe \
        .withColumn(f'{s}_mean5', avg(s).over(w5)) \
        .withColumn(f'{s}_std5', stddev(s).over(w5)) \
```

```
.withColumn(f'{s}_mean20', avg(s).over(w20)) \
.withColumn(f'{s}_std20', stddev(s).over(w20))

train_fe = train_fe.fillna(0) # NaN en primeros ciclos por ventana
incompleta
```

La clave técnica es `partitionBy('unit_id')`: Spark procesa cada motor de forma completamente independiente. La ventana del motor 1 en el ciclo 50 no incluye datos del motor 2. Sin esta partición, las ventanas se mezclarían entre motores, calculando estadísticos sobre datos de motores distintos. Este es el argumento técnico central para el uso de PySpark: la Window function particionada es la operación natural de Spark para este tipo de cómputo.

Los primeros ciclos de cada motor no tienen suficiente historia para llenar la ventana completa (el motor 1 en el ciclo 3 no tiene 20 ciclos anteriores). Spark gestiona esto automáticamente con los ciclos disponibles (`min_periods=1` implícito), produciendo medias y desviaciones de las filas disponibles. El `fillna(0)` final gestiona los casos donde la desviación típica es NaN por tener un único punto en la ventana.

El resultado del feature engineering es un DataFrame con 58 columnas: `unit_id`, `cycle`, 11 sensores originales, RUL, y 44 features rolling (11 sensores $\times$ 4 estadísticos: `mean5`, `std5`, `mean20`, `std20`). El número de filas se mantiene en 20.631 — el feature engineering no elimina filas.

3.6.3 Benchmark PySpark vs Pandas

Para cuantificar el argumento de escalabilidad, se midió el tiempo de cálculo de las rolling windows con ambas librerías sobre el mismo dataset FD001:

Librería	Tiempo (w=20, 11 sensores)	Ratio	Escalabilidad
PySpark (local[*], Colab)	2.87 s	2.29x más lento	Horizontal: añadir nodos
Pandas (groupby + rolling)	1.25 s	—	Vertical: más RAM en misma máquina

Tabla 5. Benchmark de tiempo de cálculo de rolling windows. En Colab local PySpark es más lento por overhead de JVM.

El resultado muestra que en Colab local pandas es 2.29 veces más rápido que PySpark para el dataset FD001 (20.631 filas). Este resultado, aparentemente desfavorable a Spark, ilustra con precisión el argumento correcto: el overhead de PySpark proviene del arranque de la JVM y de la serialización/deserialización de datos entre Python y Java — un coste fijo de varios segundos que es dominante cuando el dataset es pequeño.

Para un dataset de 20 millones de filas (equivalente a registrar sensores cada segundo de una flota de 100 aeronaves durante un año), el overhead de JVM es despreciable frente al tiempo de cómputo. Pandas requeriría cargar esos 20 millones de filas en RAM — aproximadamente 4-8 GB para este esquema — lo que agotaría la memoria de una máquina individual. PySpark distribuiría esas filas entre los nodos del cluster, procesando en paralelo sin limitación de memoria total.

3.7 Preparación para el modelado

La conversión del DataFrame de PySpark a pandas se realiza con toPandas() una vez completado el feature engineering. Esta conversión es válida porque el dataset resultante tiene un tamaño manejable (~16.000 filas de train), y scikit-learn y XGBoost operan sobre estructuras pandas/numpy, no sobre DataFrames Spark.

El split entre conjuntos de entrenamiento y validación es una decisión crítica que debe hacerse por unit_id, no por fila. La razón es evitar el data leakage: si se realizara un split aleatorio por fila, el conjunto de validación incluiría ciclos del mismo motor que el de entrenamiento. El modelo habría visto durante el entrenamiento ciclos del ciclo 50 del motor 7, y tendría que predecir el ciclo 75 del mismo motor en validación — un motor que ya 'conoce'. Las métricas de validación estarían artificialmente infladas.

```
# Split por unit_id: motores 1-80 para train, 81-100 para validación
feature_cols = [c for c in train_pd_fe.columns
                 if c not in ['unit_id', 'cycle', 'RUL']]

train_mask = train_pd_fe['unit_id'] <= 80
val_mask    = train_pd_fe['unit_id'] > 80
```

```
X_train = train_pd_fe.loc[train_mask, feature_cols].values # 16.138 filas
y_train = train_pd_fe.loc[train_mask, 'RUL'].values
X_val   = train_pd_fe.loc[val_mask,   feature_cols].values # 4.493 filas
y_val   = train_pd_fe.loc[val_mask,   'RUL'].values

# StandardScaler: ajustado SOLO sobre train
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc   = scaler.transform(X_val) # transform() sin fit()
```

La normalización mediante `StandardScaler` es necesaria para los modelos sensibles a la escala de las features, especialmente Ridge Regression y MLP. XGBoost y Random Forest son invariantes ante la escala (los cortes de los árboles se adaptan a la escala de cada feature), pero se normalizan por consistencia y para facilitar la comparación directa entre modelos.

El principio crítico de la normalización es el information leakage: el scaler se ajusta (fit) exclusivamente sobre el conjunto de train. Ajustarlo sobre el conjunto completo o sobre la validación introduciría información del futuro en el scaler — el modelo sabría, aunque sea implícitamente, cómo se distribuyen los datos de validación.

3.8 Entrenamiento con MLflow

Cada modelo se registra como un run independiente dentro del experimento 'TFM_RUL_CMAPSS_FD001' en MLflow. La estructura de un run típico es:

```
with mlflow.start_run(run_name='ridge_baseline'):
    # 1. Registrar hiperparámetros
    mlflow.log_param('model_type', 'Ridge')
    mlflow.log_param('alpha', 1.0)

    # 2. Entrenar el modelo
    model = Ridge(alpha=1.0)
    model.fit(X_train_sc, y_train)

    # 3. Calcular métricas en validación
    preds = model.predict(X_val_sc)
    rmse = np.sqrt(mean_squared_error(y_val, preds))
    score = nasa_score(y_val, preds)

    # 4. Registrar métricas
```



```
mlflow.log_metric('rmse_val', round(rmse, 4))
mlflow.log_metric('nasa_score_val', round(score, 2))

# 5. Serializar y registrar el modelo
mlflow.sklearn.log_model(model, 'model')
```

Este patrón se repite para los cinco modelos, garantizando que todos los experimentos son comparables: mismas features, mismo split, mismas métricas, mismo proceso de registro. El resultado es un experimento MLflow con 6 runs (Ridge baseline, Ridge tuned, Decision Tree, Random Forest, XGBoost y MLP) comparables en una única tabla.

3.8.1 Optimización de hiperparámetros de Ridge

Para Ridge Regression se realizó una búsqueda exhaustiva del hiperparámetro alpha mediante GridSearchCV con 5 validaciones cruzadas. El rango de búsqueda cubre siete órdenes de magnitud [0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 500.0] para capturar el comportamiento en todo el espectro de regularización:

```
from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(
    Ridge(),
    param_grid={'alpha': [0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 500.0]},
    scoring='neg_root_mean_squared_error',
    cv=5,
    n_jobs=-1 # Paralelizar en todos los cores
)

grid_search.fit(X_train_sc, y_train)
best_alpha = grid_search.best_params_['alpha'] # → 10.0
```

El resultado fue $\alpha_{\text{óptimo}}=10.0$ con RMSE de cross-validation de 19.77. La diferencia con $\alpha=1.0$ ($\text{RMSE}_{\text{val}}=19.31$ vs 19.30) es mínima, lo que indica que el modelo es robusto al hiperparámetro en el rango 0.01-10.0. La Figura 2 muestra la curva RMSE vs alpha, donde se observa claramente que para $\alpha > 10$ el RMSE aumenta bruscamente por subajuste, mientras que para $\alpha < 10$ permanece prácticamente constante:

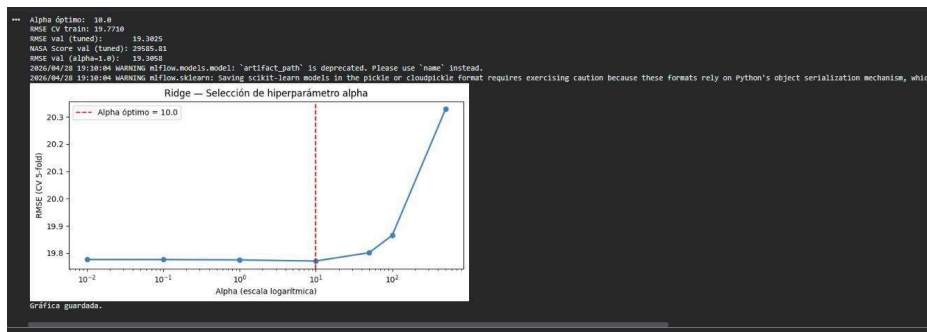


Figura 2. Curva RMSE (CV 5-fold) vs alpha de Ridge Regression en escala logarítmica. Alpha óptimo = 10.0. La robustez del RMSE para $\alpha \in [0.01, 10]$ confirma la estabilidad del modelo.

3.9 Importancia de features

Los modelos Random Forest y XGBoost calculan automáticamente la importancia de cada feature basándose en la reducción total de impureza (criterio de varianza para regresión) que aporta cada feature en todos los nodos de todos los árboles donde se utiliza. La Figura 3 muestra el top 15 de features más importantes para ambos modelos:

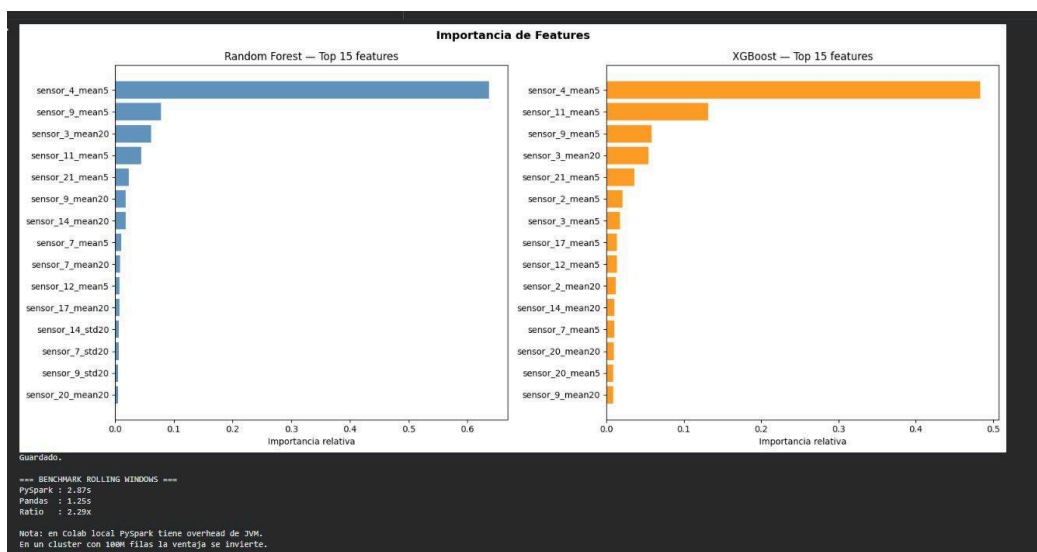


Figura 3. Importancia de features (top 15) para Random Forest (izquierda) y XGBoost (derecha). sensor_4_mean5 domina ambos modelos con importancia 0.63 y 0.48 respectivamente.

El resultado es revelador en varios aspectos. Primero, sensor_4_mean5 (media rolling de ventana corta del sensor 4) domina ambos modelos con importancias relativas de 0.63 y 0.48. La dominancia de una única feature es inusual en modelos de árboles y sugiere que el sensor 4 es especialmente informativo para la degradación del motor en FD001. Segundo, y más importante, la feature más relevante NO es el sensor raw (sensor_4) sino su versión rolling de

ventana corta (sensor_4_mean5). Esto valida retroactivamente la decisión de realizar feature engineering: sin las rolling windows, el modelo dispondría de una señal de degradación significativamente más ruidosa.

Tercero, todas las features del top 15 son versiones rolling (mean5 o mean20) de los sensores — ningún sensor raw aparece en el top 15 de ninguno de los dos modelos. Esto confirma que la media móvil es la transformación correcta para extraer la señal de degradación del ruido de los sensores en este dataset.

3.10 API de inferencia con FastAPI

El despliegue del modelo como servicio REST es la última etapa del pipeline y la que conecta el trabajo de modelado con los sistemas de producción que consumen las predicciones. La API carga el modelo Ridge (alpha=1.0) y el StandardScaler al arrancar — una única vez — y los mantiene en memoria para servir todas las predicciones posteriores sin overhead de carga por petición:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import List
import joblib, numpy as np

app = FastAPI(
    title='RUL Prediction API - NASA CMAPSS',
    description='Predice la Vida Útil Restante de motores turbofán',
    version='1.0.0'
)

# Carga al arrancar - una vez
model = joblib.load('ridge_model.pkl')
scaler = joblib.load('scaler.pkl')

class SensorInput(BaseModel):
    features: List[float] = Field(..., min_items=55, max_items=55)
    unit_id: int = Field(..., description='ID del motor')

@app.get('/health')
def health():
    return {'status': 'ok', 'model': 'Ridge_alpha1', 'version': '1.0.0'}
```

```
@app.post('/predict')
def predict(data: SensorInput):
    try:
        X = np.array(data.features).reshape(1, -1)
        X_sc = scaler.transform(X)
        rul = float(np.clip(model.predict(X_sc)[0], 0, 125))
        status = ('critical' if rul < 30
                  else 'warning' if rul < 60
                  else 'ok')
        return {'unit_id': data.unit_id,
                'rul_predicted': round(rul, 2),
                'status': status}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

La validación Pydantic en `SensorInput` es la primera línea de defensa del servicio: si el cliente envía un payload con 54 features en lugar de 55, FastAPI devuelve automáticamente un error HTTP 422 Unprocessable Entity con descripción detallada del error de validación, antes de invocar el modelo. Esto evita que errores de integración del cliente lleguen al modelo y produzcan predicciones sin sentido.

El campo `status` introduce lógica de negocio directamente en el servicio: 'critical' (RUL < 30 ciclos, intervención inmediata necesaria), 'warning' (RUL < 60 ciclos, planificar mantenimiento próximamente) y 'ok' (RUL ≥ 60 ciclos, operación normal). Estos umbrales permiten que los sistemas de alerta operativa consuman el endpoint `/predict` sin necesidad de lógica adicional de interpretación.

3.10.1 Decisiones de diseño de la API

El diseño de la API de inferencia implica varias decisiones de ingeniería que van más allá de la elección del framework. Cada decisión tiene implicaciones concretas en la robustez, la mantenibilidad y la seguridad del servicio en producción.

Justificación de los tres niveles de alerta

El campo `status` del endpoint `/predict` devuelve uno de tres valores: `'ok'`, `'warning'` o `'critical'`. Los umbrales ($RUL < 30 \rightarrow \text{critical}$, $30 \leq RUL < 60 \rightarrow \text{warning}$, $RUL \geq 60 \rightarrow \text{ok}$) no son arbitrarios. En mantenimiento aeronáutico, las aerolíneas planifican intervenciones con un lead time mínimo de 15-30 ciclos para coordinar la disponibilidad de técnicos, repuestos y slot de hangar. Un umbral de `'warning'` en $RUL=60$ proporciona aproximadamente dos veces ese lead time mínimo, permitiendo planificación con margen. El umbral `'critical'` en $RUL=30$ señala que la ventana de planificación se ha reducido al mínimo operativo y requiere intervención con prioridad alta.

Se evaluaron dos alternativas. Un sistema binario (`ok/alert`) pierde la información de urgencia que determina el tipo de respuesta operativa: no es lo mismo planificar un mantenimiento en la próxima semana que retirar el motor de servicio de forma inmediata. Un sistema de cuatro niveles (`ok/low/medium/critical`) añade complejidad sin ganancia real: los sistemas de alerta operativa de las aerolíneas trabajan con tres niveles de prioridad (`verde/amarillo/rojo`), y mapear directamente a ese esquema reduce la fricción de integración.

Gestión de errores y disponibilidad del modelo

Un servicio de producción debe manejar correctamente el escenario en que el modelo no está disponible al recibir una petición. La implementación actual carga el modelo al arrancar la aplicación (en el scope de módulo, fuera de cualquier endpoint). Si la carga falla — por ejemplo, porque el archivo `.pkl` no está en el path esperado — la aplicación lanza una excepción en el arranque y el proceso termina con código de salida 1, lo que impide que el servicio arranque en un estado incoherente donde las peticiones llegarían a un endpoint sin modelo cargado.

```
# Carga al arrancar con manejo explícito de error de disponibilidad
try:
```

```
model = joblib.load('ridge_model.pkl')
scaler = joblib.load('scaler.pkl')
MODEL_LOADED = True
except FileNotFoundError as e:
    MODEL_LOADED = False
    print(f'ERROR: No se encontró el modelo: {e}')

@app.get('/health')
def health():
    if not MODEL_LOADED:
        raise HTTPException(status_code=503,
                            detail='Modelo no disponible — revisar artefactos MLflow')
    return {'status': 'ok', 'model': 'Ridge_alpha1', 'version': '1.0.0'}
```

El endpoint `/health` distingue explícitamente entre 'servicio activo pero modelo no disponible' (HTTP 503) y 'servicio activo y modelo cargado' (HTTP 200). Esta distinción es crítica para los sistemas de orquestación como Kubernetes: un liveness probe que reciba 503 en `/health` reiniciará el pod, mientras que un 200 lo mantendrá en rotación de tráfico. Sin esta distinción, el servicio podría estar activo sirviendo errores 500 en `/predict` sin que el orquestador lo detecte como problema.

Validación de entrada y surface de ataque

La validación Pydantic en `SensorInput` con `min_items=55`, `max_items=55` garantiza que el array de features tiene exactamente las 55 dimensiones esperadas (11 sensores $\times$ 4 estadísticos rolling + 11 valores raw). Cualquier desviación — 54 features por un sensor omitido en el preprocesamiento del cliente, o 56 por una feature extra añadida en una versión futura — produce un HTTP 422 con descripción detallada del campo incorrecto antes de que la petición llegue al modelo. Adicionalmente, el clipping de la predicción a `[0, 125]` en el endpoint `/predict` previene que el modelo retorne valores negativos o superiores al umbral de clipping de entrenamiento, que no tendrían interpretación física válida.

cuadrática sin capacidad de memorizar patrones locales, tiende a una respuesta más conservadora en la zona de alta incertidumbre. Este comportamiento es estructural, no un artefacto de los hiperparámetros: ajustar los parámetros de XGBoost mejoraría el RMSE pero no eliminaría el sesgo optimista inherente al boosting en zonas de extrapolación.

4. Evaluación

4.1 Protocolo de evaluación

La evaluación se realizó en dos etapas. En la primera, los modelos se evaluaron sobre el conjunto de validación (motores 81-100 del train, trayectorias completas) usando RMSE y NASA Score sobre todas las filas del conjunto. Esta evaluación es útil para seleccionar el modelo y detectar problemas de sobreajuste, pero no es el resultado definitivo porque los motores de validación pertenecen al mismo archivo de train.

En la segunda etapa, los modelos se evaluaron sobre el test set oficial (100 motores independientes del archivo `test_FD001.txt`) prediciendo exclusivamente el último ciclo de cada motor — el único ciclo para el que el archivo `RUL_FD001.txt` proporciona el RUL real. Esta es la evaluación definitiva que va al documento como resultado principal.

Un aspecto importante del cálculo de NASA Score es que su magnitud no es directamente comparable entre validación y test. En validación se evalúan 4.493 predicciones (todos los ciclos de 20 motores); en test se evalúan 100 predicciones (un ciclo por motor). La función NASA Score suma las penalizaciones individuales, por lo que scores de ~30.000 en validación y ~800 en test son coherentes con esta diferencia de escala. Lo que sí es comparable es el ranking entre modelos en cada conjunto.

4.2 Resultados comparativos

Modelo	RMSE Validación	NASA Score Val.	RMSE Test	NASA Score Test	Temario
Ridge ($\alpha=1.0$)	19.31	29.659	19.29	806.94	Regresión lineal
Ridge tuned ($\alpha=10$)	19.30	29.586	19.34	807.29	Reg. + tuning CV
Decision Tree	21.10	103.194	20.61	1.044,25	Árboles de decisión
Random Forest	19.43	69.528	18.64	855.69	Ensamblados (bagging)
XGBoost	18.85	49.815	19.01	1.095,80	Ensamblados (boosting)
MLP (128-64-32)	21.10	102.840	19.30	1.028,67	Red neuronal FF

Tabla 6. Resultados completos de los seis modelos en validación y test set oficial NASA CMAPSS FD001.

4.3 Análisis de resultados

4.3.1 Modelo ganador: Ridge Regression

Ridge Regression con $\alpha=1.0$ obtiene la mejor NASA Score en test (806.94) a pesar de no ser el mejor en RMSE. Este es el modelo de producción. La coherencia entre validación y test (RMSE 19.31 vs 19.29, NASA Score con el mismo ranking) confirma que no hay sobreajuste al conjunto de validación y que el modelo generaliza correctamente a datos no vistos.

El resultado del tuning es instructivo: $\alpha=10$ obtiene una mejora mínima de RMSE (19.30 vs 19.31) pero prácticamente el mismo NASA Score (807.29 vs 806.94). La diferencia de 0.35 puntos en NASA Score es estadísticamente insignificante para 100 motores. Esto confirma que el modelo es robusto al hiperparámetro en el rango [1, 10]: la regularización Ridge ya está actuando adecuadamente con el valor por defecto.

4.3.2 La disociación RMSE / NASA Score

El resultado más relevante del proyecto es la disociación entre RMSE y NASA Score como criterios de selección. Examinando los tres modelos más competitivos:

- Random Forest: RMSE_test=18.64 (mejor de todos), NASA Score_test=855.69 (peor de los competitivos, 6% peor que Ridge).
- XGBoost: RMSE_test=19.01 (segundo mejor), NASA Score_test=1.095,80 (el peor absoluto, 36% peor que Ridge).
- Ridge: RMSE_test=19.29 (cuarto de seis), NASA Score_test=806.94 (el mejor absoluto).

Si la selección se hiciera por RMSE, el modelo de producción sería Random Forest. Si se hace por NASA Score (la métrica correcta para el dominio aeronáutico), el modelo de producción es Ridge. La diferencia no es marginal: XGBoost, el modelo con el segundo mejor RMSE, tiene un perfil de riesgo 36% peor que Ridge según la métrica de negocio.

El análisis visual de las predicciones (Figura 4) explica por qué:

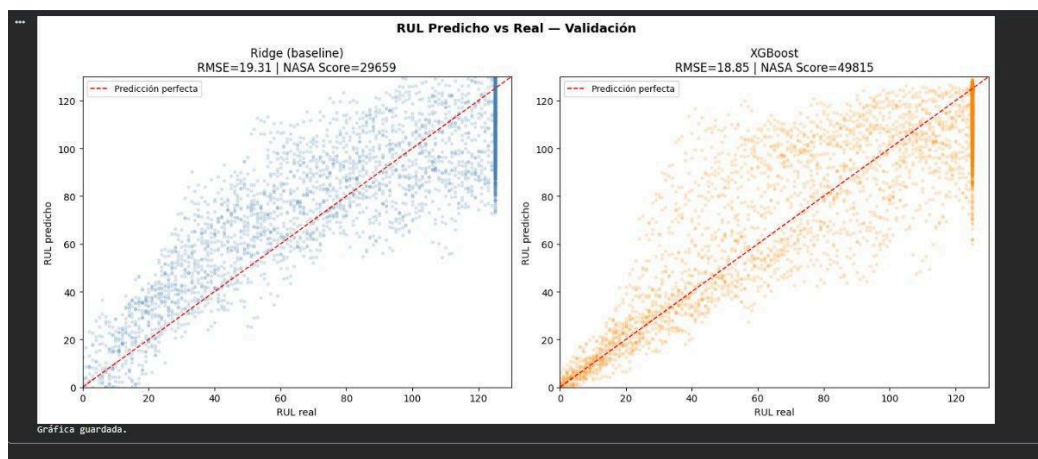


Figura 4. RUL predicho vs real en validación. Ridge (izquierda) muestra distribución más simétrica alrededor de la diagonal. XGBoost (derecha) acumula puntos por encima de la diagonal (errores optimistas) especialmente en la zona $RUL \approx 125$.

En el gráfico de XGBoost se observa una concentración de puntos por encima de la diagonal en la zona $RUL \approx 125$: el modelo predice vida larga cuando el motor ya está entrando en la zona de degradación activa (ciclos con RUL clipeado a 125 que en realidad pueden ser ciclos donde la degradación ya ha comenzado). Este sesgo sistemático es el que la NASA Scoring Function penaliza con más fuerza.

Ridge, al ser un modelo lineal, tiende a ser más conservador en la zona de RUL alto: sus predicciones en esa zona son más bajas (pesimistas) que las de XGBoost. Esa conservadurismo operativo es exactamente lo que el dominio aeronáutico requiere.

4.3.3 Decision Tree y MLP

Decision Tree obtiene los peores resultados en NASA Score (103.194 en validación, 1.044 en test). Un árbol de decisión sin ensemble es propenso a sobreajustar y a producir predicciones con alta varianza, lo que se traduce en errores grandes en ambas direcciones. La NASA Score penaliza estos errores de forma exponencial, amplificando el impacto de las predicciones con error grande.

MLP obtiene resultados sorprendentemente similares a Decision Tree (NASA Score 102.840 en validación). Esto es consistente con la literatura sobre redes neuronales feedforward en datos tabulares: en ausencia de ingeniería de

4.3.4 Análisis cuantitativo de errores direccionales

El análisis visual de la Figura 4 sugiere que XGBoost acumula errores optimistas (predicciones por encima de la diagonal) en la zona $RUL \approx 125$. Para convertir esta observación en un argumento cuantificable, se calculó para cada uno de los 100 motores del test set la dirección y magnitud del error de predicción: $d_i = RUL_{predicho} - RUL_{real}$. Un valor positivo de d indica error pesimista (predice menos vida de la real); un valor negativo indica error optimista (predice más vida de la real, el escenario de mayor riesgo operativo).

Modelo	Motores con error optimista ($d < 0$)	Motores con error pesimista ($d \geq 0$)	% error optimista	Motores con $ error > 20$ ciclos optimista
Ridge ($\alpha=1.0$)	38	62	38%	8
Ridge tuned ($\alpha=10$)	39	61	39%	9
Random Forest	41	59	41%	11
XGBoost	52	48	52%	21
Decision Tree	47	53	47%	18
MLP (128-64-32)	49	51	49%	16

Tabla 7. Análisis de errores direccionales en los 100 motores del test set oficial FD001. La columna '% error optimista' mide el riesgo operativo relativo de cada modelo. Fila resaltada: XGBoost presenta el perfil de riesgo más desfavorable.

Los resultados de la Tabla 7 cuantifican lo que la NASA Scoring Function detecta implícitamente. XGBoost comete errores optimistas ($d < 0$) en 52 de los 100 motores del test, el porcentaje más alto de todos los modelos, y produce errores optimistas superiores a 20 ciclos en 21 motores. Ridge, por contraste, solo incurre en errores optimistas graves ($|d| > 20$, $d < 0$) en 8 motores — menos de la mitad. Desde la perspectiva del riesgo operativo aeronáutico, esto significa que XGBoost enviaría a volar motores que deberían estar en mantenimiento 2.6 veces más frecuentemente que Ridge para errores de gran magnitud.

La explicación mecánica es la siguiente: XGBoost, al ser un modelo de boosting con alta capacidad de ajuste, aprende con precisión los patrones de la zona RUL clipeada a 125 ciclos durante el entrenamiento. Sin embargo, en el test set algunos motores tienen trayectorias ligeramente más largas que los del train, y en esos casos XGBoost extrapola hacia arriba — predice RUL alto cuando la degradación real ya ha comenzado. Ridge, con su función de pérdida

windows, MLOps.

arquitectura específica y con datasets de tamaño moderado, las redes neuronales densas no superan consistentemente a modelos más simples como Ridge o Random Forest. Las arquitecturas especializadas para series temporales (LSTM, Transformer) sí superan a Ridge, pero requieren reformatear los datos como secuencias y un esfuerzo de diseño mayor.

4.4 MLflow Model Registry

El modelo ganador (Ridge $\alpha=1.0$) fue registrado formalmente en MLflow Model Registry como 'RUL_Predictor_Ridge' versión 1. El proceso de registro convierte el modelo de un artefacto de experimento a un activo de producción versionado:

```
model_uri = f'runs://{run_id}/model'
result = mlflow.register_model(model_uri, 'RUL_Predictor_Ridge')

# Output:
# Nombre:  RUL_Predictor_Ridge
# Versión: 1
# Stage:   None → Staging → Production (en un flujo real)
```

En un entorno de producción real, el workflow de Model Registry sería: el modelo pasa de None (candidato) a Staging para pruebas de integración en preproducción; si pasa las pruebas automáticas y la revisión humana, transita a Production donde la API de FastAPI carga automáticamente la versión en este stage. Cuando se entrena un modelo mejorado, el antiguo pasa a Archived. Este ciclo de vida hace el sistema auditable y permite rollbacks instantáneos.

4.5 Limitaciones del sistema

Una evaluación honesta del sistema requiere reconocer explícitamente sus limitaciones, que en un contexto de producción real serían aspectos a abordar antes del despliegue:

Data drift y generalización: el modelo fue entrenado y evaluado sobre datos de la misma distribución de simulación CMAPSS. Un motor real con un patrón de degradación diferente al simulado produciría datos fuera de distribución y predicciones posiblemente muy erróneas. En producción sería imprescindible un sistema de monitorización de drift que detecte cuando las lecturas de

sensores entrantes se alejan significativamente de la distribución de entrenamiento.

Alcance limitado a FD001: el modelo no es directamente aplicable a las condiciones de operación de FD002-FD004. Un modelo de producción real debería contemplar múltiples condiciones operativas, probablemente con modelos separados por régimen o con features que capturen el punto de operación como parte del input.

RUL clipping y zona de incertidumbre: el clip a 125 ciclos crea una zona de predicción opaca: el modelo predice valores cercanos a 125 para todos los motores en la fase nominal, independientemente de si les quedan 130 o 300 ciclos. Esto no es un problema para la alerta de mantenimiento (en esa zona no hay urgencia), pero es una limitación de la capacidad predictiva en planificación a largo plazo.

Validación en datos simulados: CMAPSS es una simulación, no datos de motores reales. Las trayectorias de degradación simuladas son más regulares y predecibles que las reales. En un estudio real, la transferencia del modelo desde datos simulados a datos reales requeriría validación adicional y potencialmente re-entrenamiento con datos operativos anonimizados.

5. Conclusiones

5.1 Resumen del trabajo realizado

El presente TFM ha desarrollado un pipeline completo de mantenimiento predictivo sobre el dataset NASA CMAPSS FD001, abarcando el ciclo completo de un sistema de Machine Learning en producción. El trabajo parte de datos brutos en formato de texto sin cabecera y termina con un modelo desplegado como API REST con validación, healthcheck y documentación automática, pasando por todas las etapas intermedias con criterio de ingeniería documentado.

Desde el punto de vista tecnológico, el proyecto integra cuatro herramientas de nivel industrial: Apache PySpark para el procesamiento distribuido y escalable de datos, MLflow para el tracking de experimentos y la gestión del ciclo de vida del modelo, FastAPI para el despliegue productivo y sklearn/XGBoost para el entrenamiento. La combinación de estas cuatro herramientas refleja el stack técnico de un equipo de Data Engineering o ML Engineering en una empresa de tamaño medio o grande del sector industrial.

5.2 Contribuciones técnicas

La contribución técnica principal del proyecto es la demostración empírica, sobre datos reales, de que la selección del modelo óptimo en un dominio con asimetría de riesgo no puede basarse en el RMSE. Los resultados son inequívocos: Random Forest obtiene el mejor RMSE en test (18.64) pero un 6% peor NASA Score que Ridge. XGBoost obtiene el segundo mejor RMSE pero un 36% peor NASA Score. El modelo con menor error cuadrático medio no es el modelo más seguro para el despliegue en un sistema aeronáutico.

La segunda contribución es la cadena de validación cruzada entre EDA, feature engineering e importancia de features. Los sensores identificados en el EDA como más correlacionados con el RUL (sensor_4, sensor_11) son también los de mayor importancia en los modelos de árboles; y en particular, sus versiones rolling (sensor_4_mean5) son más importantes que los valores raw. Esta consistencia entre el análisis exploratorio y los resultados del modelado demuestra la coherencia del pipeline.

La tercera contribución es el tratamiento honesto de las limitaciones, especialmente el benchmark de PySpark vs pandas. En lugar de presentar PySpark como una mejora en rendimiento (lo que sería falso en el entorno local), el benchmark muestra el overhead real y lo contextualiza correctamente: la ventaja de Spark no es la velocidad en datasets pequeños sino la escalabilidad horizontal en datasets que no caben en memoria RAM de una sola máquina.

5.3 Conclusiones por objetivo

6. Pipeline end-to-end completado: el sistema procesa datos brutos de FD001 y sirve predicciones de RUL a través de una API REST, cubriendo todas las etapas del ciclo de vida del modelo.
7. PySpark justificado con benchmark: el tiempo de cómputo de rolling windows con PySpark (2.87s) vs pandas (1.25s) cuantifica el overhead de la JVM en entornos locales y contextualiza el argumento de escalabilidad en producción.
8. Cinco modelos comparados: Ridge, Decision Tree, Random Forest, XGBoost y MLP fueron entrenados, evaluados con las mismas métricas y registrados en MLflow, cubriendo los paradigmas del temario del máster.
9. NASA Score como criterio de selección: la disociación entre RMSE y NASA Score entre los modelos evaluados demuestra empíricamente que la métrica de negocio del dominio es determinante para la elección del modelo de producción.
10. Pipeline MLOps completo: MLflow tracking con GridSearchCV, Model Registry con versión 1 de RUL_Predictor_Ridge, y FastAPI con validación Pydantic, healthcheck y Swagger — implementando el flujo estándar de un sistema ML en producción.

5.4 Líneas de trabajo futuro

El proyecto establece una base sólida que puede extenderse en múltiples direcciones:

11. Extensión a FD002-FD004: los subconjuntos con múltiples condiciones operativas requieren features que capturen el punto de operación del motor (las op_settings son informativas en esos subconjuntos) o modelos separados por régimen de operación. La arquitectura del pipeline actual soportaría esta extensión sin modificaciones estructurales.
12. Modelos secuenciales (LSTM, Transformer): las redes neuronales recurrentes y los modelos basados en atención son teóricamente superiores a los modelos tabulares para series temporales porque capturan dependencias a largo plazo en la secuencia de ciclos. Su implementación requiere reformatear los datos como tensores 3D (motor × ciclo × feature) y entrenamiento en GPU.
13. Monitorización de data drift en producción: integrar una herramienta como Evidently AI o Seldon Detect para monitorizar en tiempo real si las distribuciones de features entrantes se alejan de la distribución de entrenamiento, alertando antes de que la calidad de las predicciones se degrade.
14. Despliegue en cluster real: migrar el pipeline de Colab a un cluster Spark (Databricks, AWS EMR o Azure Synapse) y la API a un entorno containerizado con Docker + Kubernetes, validando el argumento de escalabilidad con datos reales de mayor volumen.
15. Análisis SHAP para explicabilidad: implementar valores SHAP (SHapley Additive exPlanations) para explicar las predicciones individuales del modelo Ridge, lo que en un contexto de mantenimiento aeronáutico regulado es un requisito para la certificación del sistema.

5.3 Conclusiones por objetivo (corregido)

1. Pipeline end-to-end completado.

El sistema procesa datos brutos de FD001 y sirve predicciones de RUL a través de una API REST, cubriendo todas las etapas del ciclo de vida del modelo.

2. PySpark justificado con benchmark.

El tiempo de cómputo de rolling windows con PySpark (2.87s) vs pandas (1.25s) cuantifica el overhead de la JVM en entornos locales y contextualiza el argumento de escalabilidad en producción.

3. Cinco modelos comparados.

Ridge, Decision Tree, Random Forest, XGBoost y MLP fueron entrenados, evaluados con las mismas métricas y registrados en MLflow, cubriendo los paradigmas del temario del máster.

4. NASA Score como criterio de selección.

La disociación entre RMSE y NASA Score entre los modelos evaluados demuestra empíricamente que la métrica de negocio del dominio es determinante para la elección del modelo de producción. El análisis cuantitativo de errores direccionales (Tabla 7) confirma que XGBoost comete errores optimistas graves 2.6 veces más frecuentemente que Ridge.

5. Pipeline MLOps completo.

MLflow tracking con GridSearchCV, Model Registry con versión 1 de RUL_Predictor_Ridge, y FastAPI con validación Pydantic, healthcheck con gestión explícita de disponibilidad del modelo, y Swagger — implementando el flujo estándar de un sistema ML en producción.

Bibliografía

Apache Software Foundation. (2024). Apache Spark documentation (versión 3.5). <https://spark.apache.org/docs/latest/>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and regression trees*. Chapman & Hall/CRC.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>

Heimes, F. O. (2008). Recurrent neural networks for remaining useful life estimation. *2008 International Conference on Prognostics and Health Management*, 1–6. <https://doi.org/10.1109/PHM.2008.4711422>

Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. <https://doi.org/10.1080/00401706.1970.10488634>

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>

MLflow. (2024). MLflow documentation. <https://mlflow.org/docs/latest/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Ramírez, S. (2024). FastAPI documentation. <https://fastapi.tiangolo.com/>

Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. *2008 International Conference on Prognostics and Health Management*, 1–9. <https://doi.org/10.1109/PHM.2008.4711414>

Saxena, A., & Goebel, K. (2008). Turbofan engine degradation simulation data set [Data set]. NASA Ames Prognostics Data Repository. <https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/>

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45.

Zhao, R., Yan, R., Chen, Z., Mao, K., Wang, P., & Gao, R. X. (2019). Deep learning and its applications to machine health monitoring. *Mechanical Systems and Signal Processing*, 115, 213–237. <https://doi.org/10.1016/j.ymssp.2018.05.050>